Assignment 4

a. Gemini and CoPilot
b. Searched both up in a Google query
c. I uploaded the six .txt files (puzzle1, puzzle2, puzzle3, puzzle4, puzzle5, and solution1), as well as this prompt:

Use two GenAI's to generate C++ code using objects, not functions, to implement the following requirements (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):
• Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
• A typical puzzle is shown here (image credit: Wikimedia Commons):
o
• Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
o Each digit from 1 to 9 must appear exactly once in each row.
o Each digit from 1 to 9 must appear exactly once in each column.
o Each digit from 1 to 9 must appear exactly once in each block.
• Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
• The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
• It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.
• That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
• Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
• This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively

try all possibilities.
• You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
• Write a recursive object-oriented program to find the solution to the five puzzle files.
• The output for each solution, should include:
o The puzzle file name (e.g., puzzle1.txt)
o The puzzle from the file printed out.
o The solution to the puzzle printed out or if no solution is found, "No solution found" printed out.
• The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
• To help you debug your program, solution1.txt is a solution to puzzle1.txt.

d. Here is the **Gemini** code:

```
#include <iostream>
#include <fstream>
#include <string>

class Board {
private:
   int grid[9][9];

public:
   Board() {
      for (int i = 0; i < 9; ++i) {
         for (int j = 0; j < 9; ++j) {
            grid[i][j] = 0;
         }
      }
   }

   bool loadFromFile(const std::string& filename) {
      std::ifstream file(filename);
      if (!file.is_open()) return false;
      for (int i = 0; i < 9; ++i) {
         for (int j = 0; j < 9; ++j) {
            char ch;
            if (!(file >> ch)) return false;
            if (ch == '_') {
               grid[i][j] = 0;
            } else {
               grid[i][j] = ch - '0';
            }
         }
      }
      return true;
   }

   void print() const {
      for (int i = 0; i < 9; ++i) {
```

```
            for (int j = 0; j < 9; ++j) {
                if (grid[i][j] == 0) {
                    std::cout << "_ ";
                } else {
                    std::cout << grid[i][j] << " ";
                }
            }
            std::cout << "\n";
        }
    }

    int getCell(int r, int c) const {
        return grid[r][c];
    }

    void setCell(int r, int c, int val) {
        grid[r][c] = val;
    }

    bool isValid(int row, int col, int num) const {
        for (int i = 0; i < 9; ++i) {
            if (grid[row][i] == num && i != col) return false;
            if (grid[i][col] == num && i != row) return false;
        }
        int startRow = row - row % 3;
        int startCol = col - col % 3;
        for (int i = 0; i < 3; ++i) {
            for (int j = 0; j < 3; ++j) {
                int r = startRow + i;
                int c = startCol + j;
                if (grid[r][c] == num && (r != row || c != col)) return false;
            }
        }
        return true;
    }

    bool findEmpty(int& row, int& col) const {
        for (row = 0; row < 9; ++row) {
            for (col = 0; col < 9; ++col) {
                if (grid[row][col] == 0) return true;
            }
        }
        return false;
    }
};

class SudokuSolver {
private:
    Board board;
    int solutionCount;

    void solveRecursive() {
        int row, col;
        if (!board.findEmpty(row, col)) {
```

```
            solutionCount++;
            std::cout << "Solution " << solutionCount << ":\n";
            board.print();
            std::cout << "\n";
            return;
        }

        for (int num = 1; num <= 9; ++num) {
            if (board.isValid(row, col, num)) {
                board.setCell(row, col, num);
                solveRecursive();
                board.setCell(row, col, 0); // Backtrack
            }
        }
    }

public:
    explicit SudokuSolver(const Board& b) : board(b), solutionCount(0) {}

    void solve(const std::string& filename) {
        std::cout << "========================================\n";
        std::cout << "Puzzle File: " << filename << "\n";
        std::cout << "Initial Puzzle:\n";
        board.print();
        std::cout << "\nSolutions:\n";
        solveRecursive();
        if (solutionCount == 0) {
            std::cout << "No solution found\n";
        } else {
            std::cout << "Total solutions found: " << solutionCount << "\n";
        }
        std::cout << "========================================\n\n";
    }
};

class Application {
public:
    void run() {
        std::string files[] = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", "puzzle5.txt"};
        for (const auto& file : files) {
            Board board;
            if (board.loadFromFile(file)) {
                SudokuSolver solver(board);
                solver.solve(file);
            } else {
                std::cout << "Could not open file: " << file << "\n";
            }
        }
    }
};

int main() {
    Application app;
    app.run();
```

```
    return 0;
}
```

And here is the **CoPilot** code:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

class Sudoku {
private:
    vector<vector<char>> grid;

public:
    Sudoku() {
        grid.resize(9, vector<char>(9));
    }

    bool loadPuzzle(const string& filename) {
        ifstream fin(filename);

        if (!fin) {
            cout << "Error opening file: " << filename << endl;
            return false;
        }

        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                fin >> grid[row][col];
            }
        }

        fin.close();
        return true;
    }

    void printBoard() const {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                cout << grid[row][col] << " ";
            }
            cout << endl;
        }
    }

    bool findEmptyCell(int& row, int& col) const {

        for (row = 0; row < 9; row++) {
            for (col = 0; col < 9; col++) {
```

```
            if (grid[row][col] == '_')
                return true;
        }
    }

    return false;
}

bool validInRow(int row, char value) const {

    for (int col = 0; col < 9; col++) {
        if (grid[row][col] == value)
            return false;
    }

    return true;
}

bool validInColumn(int col, char value) const {

    for (int row = 0; row < 9; row++) {
        if (grid[row][col] == value)
            return false;
    }

    return true;
}

bool validInBlock(int row, int col, char value) const {

    int startRow = (row / 3) * 3;
    int startCol = (col / 3) * 3;

    for (int r = startRow; r < startRow + 3; r++) {
        for (int c = startCol; c < startCol + 3; c++) {

            if (grid[r][c] == value)
                return false;
        }
    }

    return true;
}

bool isValidMove(int row, int col, char value) const {

    return validInRow(row, value) &&
           validInColumn(col, value) &&
           validInBlock(row, col, value);
}

void setValue(int row, int col, char value) {
    grid[row][col] = value;
}
```

```
    vector<vector<char>> getGrid() const {
        return grid;
    }
};

class SudokuSolver {
private:
    vector<vector<vector<char>>> solutions;

public:

    void solve(Sudoku& puzzle) {

        int row;
        int col;

        if (!puzzle.findEmptyCell(row, col)) {

            solutions.push_back(puzzle.getGrid());
            return;
        }

        for (char digit = '1'; digit <= '9'; digit++) {

            if (puzzle.isValidMove(row, col, digit)) {

                puzzle.setValue(row, col, digit);

                solve(puzzle);

                // Backtracking step
                puzzle.setValue(row, col, '_');
            }
        }
    }

    void printSolutions() const {

        if (solutions.empty()) {

            cout << "\nNo solution found.\n";
            return;
        }

        for (size_t i = 0; i < solutions.size(); i++) {

            cout << "\nSolution " << i + 1 << ":\n\n";

            for (int row = 0; row < 9; row++) {

                for (int col = 0; col < 9; col++) {

                    cout << solutions[i][row][col] << " ";
```

```
            }

            cout << endl;
        }
    }

    cout << "\nSolutions found: "
         << solutions.size()
         << endl;
  }
};

class PuzzleManager {
private:
  vector<string> puzzleFiles;

public:

  PuzzleManager() {

    puzzleFiles.push_back("puzzle1.txt");
    puzzleFiles.push_back("puzzle2.txt");
    puzzleFiles.push_back("puzzle3.txt");
    puzzleFiles.push_back("puzzle4.txt");
    puzzleFiles.push_back("puzzle5.txt");
  }

  void processPuzzles() {

    for (string filename : puzzleFiles) {

      Sudoku puzzle;

      if (!puzzle.loadPuzzle(filename))
        continue;

      cout << "\n================================================";
      cout << "\nPuzzle File: " << filename;
      cout << "\n================================================\n";

      cout << "\nOriginal Puzzle:\n\n";
      puzzle.printBoard();

      SudokuSolver solver;
      solver.solve(puzzle);

      cout << "\nSolved Puzzle(s):\n";
      solver.printSolutions();

      cout << "\n";
    }
  }
};
```

```
int main() {

    PuzzleManager manager;

    manager.processPuzzles();

    return 0;
}
```

e. Analysis
   i. **Correctness:** Both programs are created with object-orientation and do not use functions. Since the specifications state that the only inputs are the 5 puzzle files, neither program needs to account for any other inputs. They handle the 5 files correctly, and print according to the requirements. Both include breaks if they cannot find the file or open a file. Both use recursion to accomplish the goals. Neither program crashes randomly. Both programs correctly find the solutions to the puzzles.
   ii. **Execution time:** Since both programs utilize checkers to determine if a move is valid, the worst case Big O for both programs is O(9^n), which is determined on how many empty cells there are in the puzzles. However, because of the recursive nature of the programs, the overall Big O is much lower. This is due to the validity checkers immediately stopping several pathways of moves from occurring. Overall though, both programs have the same execution time.
   iii. **Space complexity:** Because the puzzle files have a set 9x9 orientation, that has a space complexity of O(1) because it does not fluctuate. Storing the solutions has a space complexity of O(n) where n is the number of solutions for the puzzles. However, since both programs are solving the same puzzles, they have the same amount of solutions. Overall, they have the same space complexity.
   iv. **Maintainability:** Both programs can handle edge cases. Both programs have little to zero comments explaining the code. However, CoPilot's code has better variable names and writes its code in a way that makes more sense, as compared to the Gemini code. Because of this, CoPilot's maintainability is better.

f. I will be choosing CoPilot's code because both programs have the same correctness, by taking an object-oriented approach and using recursiveness, as well as having the same execution time and space complexity. However, CoPilot has better maintainability and is overall easier to read than the Gemini code.
g. To improve the code, I put comments explaining blocks of code and what each class does in order to improve maintainability. I also changed **grid.resize(9, vector<char>(9));** to **grid.resize(9, vector<char>(9, '_'));** to avoid using null characters and to simulate the Sudoku cells better, which improves space complexity.